Program Synthesis using Conflict-Driven Learning

Varun Varma

September 29th, 2020

Abstract

Program synthesis is the problem of generating a program from a given specification. A common technique in this field is enumerating through the search space. A major shortcoming of this approach is that it does not learn from past mistakes. This paper introduces a new conflict driven synthesis algorithm that can learn from its past mistakes for more efficient synthesis.

The conflict driven synthesis algorithm starts with a specification and an empty knowledge base. The algorithm determines a hole to fill in the program and replaces it with a new term that satisfies the constraints of the knowledge base using a combination of constraint solving and heuristics. If the new partial program has a conflict, the root cause of the conflict is identified, encoded as a SAT propositional formula, and added to the knowledge base to prevent further errors. A novel technique introduced in the paper called *equivalent modulo conflict*, where given a partial program with a conflicting component X the algorithm finds other components that result in the same conflict as X, is used to determine the root cause of a conflict. The algorithm then backtracks and continues enumeration with its updated knowledge base. Synthesis will terminate if the constraints in the knowledge base are unsatisfiable or if a solution is found.

A prototype implementation of the conflict driven synthesis algorithm, names *NEO*, was constructed. *NEO* was benchmarked against state-of-the-art program synthesis engines *MORPHEUS* and *DEEPCODER*, where it performed significantly better, especially on more difficult problems. In addition, *NEO* was benchmarked against a version of itself that did not perform conflict analysis and was shown to have significant performance gains (an average of 19.8 times faster in the hard category), which suggest that conflict analysis is beneficial for program synthesis (Feng, Martins, Bastani, & Dillig, 2018).

Materials and Methods

A prototype of the algorithm outlined in the paper, called *NEO*, was implemented in Java. *NEO* and a version of *NEO* without conflict analysis where tested on benchmarks from the *MORPHEAUS* and *DEEPCODER* program synthesis engines. The total synthesis time was recorded for each benchmark. All benchmarks were run on Intel Xeon(R) computer with an E5-2640 v3 CPU, 32GB of memory, running the Ubuntu 16.04 operating system, and using a timeout of 5 minutes (Feng, Martins, Bastani, & Dillig, 2018).

Analysis

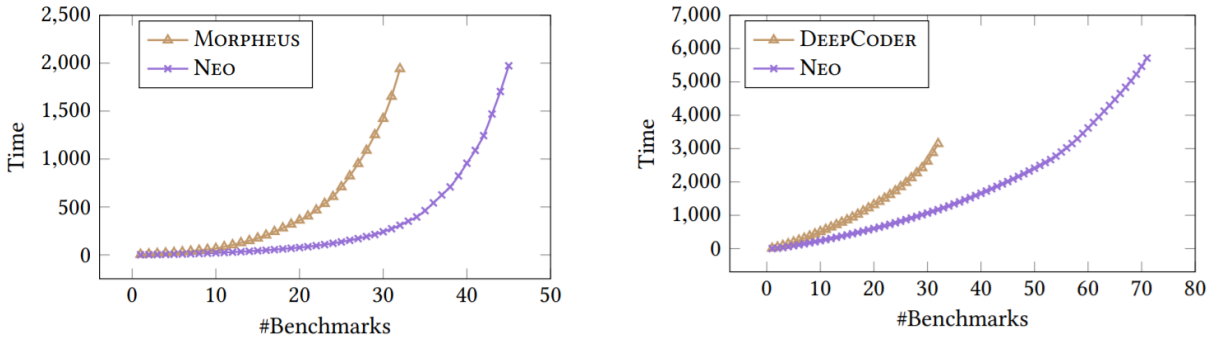

Figure 1 – Comparison of NEO *and* MORPHEAUS *(Left), Comparison of* NEO *and* DEEPCODER *(Right)*

Figure 1 shows a comparison of *NEO* and two other program synthesis engines. Time is represented on the Y axis and benchmark difficulty is represented on the X axis. A lower time is better. The other two program synthesis engines were trained only on 30 of the 50 benchmarks *NEO* was trained on. NEO is shown to be considerably faster than the other state, especially on more difficult tasks, which suggests that conflict analysis produces major efficiency gains.

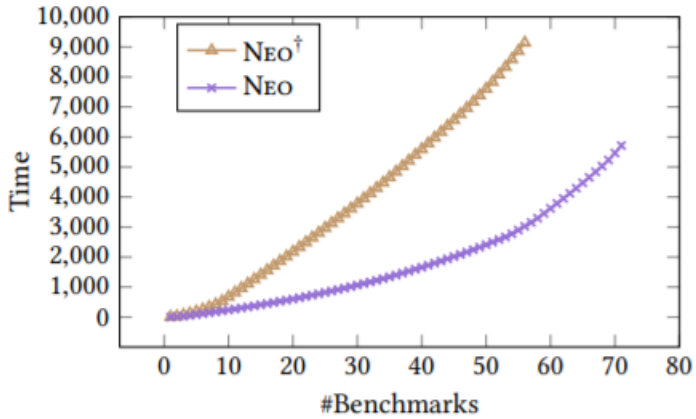

Figure 2 – Comparison of NEO *and a version of* NEO *without conflict analysis (*NEO+*)*

Figure 2 shows a comparison of *NEO* and a version of *NEO* without conflict analysis. Time is represented on the Y axis and benchmark difficulty is represented on the X axis. A lower time is better. The version of *NEO* with conflict analysis is shown to perform significantly than the version without, which suggests that the high performance of *NEO* compared to the state of the art is from the addition of conflict analysis and not some other optimization.

My paper requires the use of refinement types as program specifications. These refinement types need to be recursively decomposed during synthesis, which produces the partial functions necessary for the conflict driven synthesis algorithm. The type checking benefits from using refinement types can help speed up conflict analysis by potentially replacing the *equivalent modulo conflict* step of the algorithm. This will allow further pruning of the search space and consequently, faster synthesis times.

Bibliography

Feng, Y., Martins, R., Bastani, O., & Dillig, I. (2018). Program synthesis using conflict-driven learning. *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation - PLDI 2018*. doi:10.1145/3192366.3192382